

INTERAÇÃO COM MODELOS DE GRANDE ESCALA

Trabalho Prático #2

Retail Vision Intelligence System

Cotação: 6,5 valores / 20

Ano Letivo 2025/2026

1. Contexto e Motivação

O Projeto 1 transformou dados de detecção brutos em inteligência operacional: percursos de clientes, padrões de afluência, anomalias de tráfego e relatórios semanais em linguagem natural. Toda essa inteligência foi construída sobre dados estruturados, isto é: sequências de eventos com timestamps e identificadores de zona.

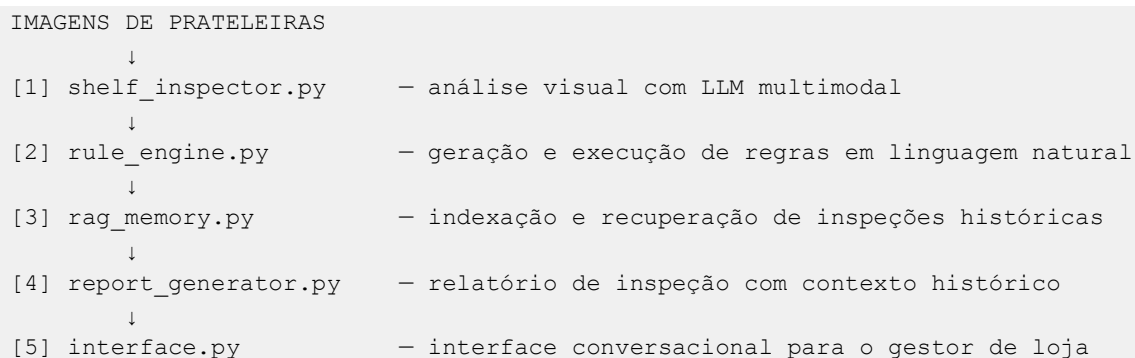
Existe, no entanto, uma dimensão da operação de uma loja de retalho que os dados de evento não capturam: o estado físico das prateleiras. Uma zona pode ter afluência elevada e *dwell time* longo, mas se o produto estiver mal posicionado, se uma prateleira estiver vazia, ou se o planograma não estiver a ser cumprido, a loja está a perder vendas de forma invisível para qualquer sistema baseado apenas em detecção de presença.

Este projeto constrói a camada de inteligência visual que falta. O sistema recebe imagens de prateleiras, analisa-as com modelos de linguagem multimodais, deteta problemas operacionais, e permite que um gestor de loja defina regras de detecção em linguagem natural que o sistema converte automaticamente em configurações executáveis. O histórico de inspeções é indexado numa base de dados vetorial, permitindo recuperação semântica de padrões passados para contextualizar análises futuras.

O produto final é um sistema de inspeção contínua de prateleiras com memória, capaz de aprender as regras do gestor e de integrar a sua análise visual com os dados de trajetória do Projeto 1.

2. Descrição do Sistema

O sistema tem cinco componentes principais que o aluno deve implementar e integrar:



Cada componente deve ser executável de forma independente, com input e output bem definidos. O sistema completo é orquestrado por `interface.py`, que expõe uma interface conversacional onde o gestor pode submeter imagens, fazer perguntas e definir novas regras.

3. Dataset de Imagens

O aluno deve construir o seu próprio dataset a partir de fontes públicas como datasets de retalho da NVIDIA. Esta é uma decisão técnica que deve ser justificada no relatório.

3.1 Fontes Recomendadas

- SKU-110K (Goldman et al., 2019) — 11.762 imagens de supermercado com anotações de bounding box. https://github.com/eg4000/SKU110K_CVPR19
- Grocery Store Dataset (Hult et al., 2019) — imagens de produtos e prateleiras naturais. HuggingFace: [johnanvik/grocery-store-dataset](https://huggingface.co/datasets/johnanvik/grocery-store-dataset)
- GroZi-120 — 120 produtos de supermercado em condições naturais de prateleira.
- Open Images Dataset (Google) — filtrado por categorias relevantes (supermarket shelf, retail).
- Imagens próprias recolhidas pelo aluno em supermercados locais.

3.2 Requisitos Mínimos

O dataset de trabalho deve conter no mínimo 500 imagens distribuídas da seguinte forma:

Tipo de imagem	Mínimo	Descrição
Prateleira normal	150	Produto bem posicionado, sem problemas visíveis
Prateleira vazia (total ou parcial)	100	Uma ou mais posições sem produto
Violação de planograma	100	Produto na posição errada, etiqueta ausente, produto tombado
Prateleira suja / desordenada	80	Produto desalinhado, embalagens danificadas
Caso ambíguo	70	Situações onde a classificação não é óbvia

O aluno deve documentar a origem de cada imagem e garantir que o uso respeita os termos de licença das fontes. Durante a avaliação, o sistema será testado em imagens adicionais não vistas durante o desenvolvimento.

4. Componente 1: Shelf Inspector

4.1 Modelo

O sistema usa Google Gemini 1.5 Flash através da API gratuita (Google AI Studio — sem cartão de crédito, 15 req/min, 1500 req/dia). O aluno deve registar-se em <https://aistudio.google.com> e obter uma chave de API gratuita.

O uso de outros modelos multimodais com API gratuita é permitido (e.g., HuggingFace Inference API com LLaVA-Next, OpenAI GPT-4o free tier), mas o aluno deve justificar a escolha e documentar os limites do modelo.

4.2 Análise Visual — Output Estruturado

Para cada imagem submetida, o sistema deve produzir uma análise estruturada em JSON com o seguinte schema obrigatório:

```
{
  "inspection_id": "INS_20250317_143022_001",
  "timestamp": "2025-03-17T14:30:22Z",
  "image_path": "path/to/image.jpg",
  "zone_id": "Z_S3",
  "overall_status": "ok|warning|critical",
```

```
"issues": [
  {
    "issue_id": "ISS_001",
    "type": "empty_shelf|wrong_product|damaged|misaligned|label_missing|other",
    "location": "e.g. prateleira inferior, lado esquerdo",
    "severity": "low|medium|high",
    "description": "descrição em linguagem natural do problema",
    "confidence": 0.0,
    "affected_area_pct": 0.0
  }
],
"shelf_fill_rate": 0.0,
"products_detected": ["lista de categorias de produto visíveis"],
"model_reasoning": "cadeia de raciocínio explícita antes da classificação"
}
```

O campo **model_reasoning** é obrigatório e deve conter a cadeia de raciocínio explícita do modelo antes da classificação final. O aluno deve usar *prompting* que force o modelo a raciocinar passo a passo antes de produzir o JSON. Este campo é usado na avaliação para verificar se o modelo está a raciocinar correctamente ou a produzir outputs plausíveis sem fundamento.

4.3 Três Estratégias de Prompting Obrigatórias

O aluno deve implementar e comparar obrigatoriamente três estratégias de prompting para a análise visual:

- Estratégia A - Zero-shot direto: instrução directa pedindo a análise e o JSON, sem exemplos nem estrutura de raciocínio.
- Estratégia B - Chain-of-Thought visual: o modelo é forçado a raciocinar explicitamente sobre regiões da imagem antes de classificar. O prompt guia por etapas: descrever o que vê, identificar anomalias zona a zona, classificar cada anomalia, e só então produzir o JSON.
- Estratégia C - Few-shot com exemplos textuais: dado que passar múltiplas imagens por chamada consome quota de API, esta estratégia usa descrições textuais de análises anteriores correctas como exemplos few-shot antes da imagem a analisar.

A comparação das três estratégias deve ser feita sobre um conjunto de pelo menos 15 imagens com ground truth definido pelo aluno, usando as métricas da Secção 9.

4.4 Gestão de Limites de API

Com 1500 req/dia gratuitas no Gemini 1.5 Flash, o aluno deve implementar obrigatoriamente:

- *Cache* local de resultados: inspeções já realizadas não devem consumir quota adicional - guardar o resultado em disco e retornar do cache se a imagem não foi modificada (verificar via hash MD5 do ficheiro).
- *Rate limiting*: respeitar o limite de 15 req/min com backoff exponencial em caso de erro 429.
- *Fallback* gracioso: se a quota diária for esgotada, o sistema deve continuar a funcionar para imagens em cache e notificar claramente quando não consegue processar novas imagens.

5. Componente 2: Rule Engine

5.1 O Problema

O gestor de loja não fala JSON. Fala português, tem intuições sobre o que deve e não deve acontecer nas suas prateleiras, e quer ser notificado quando essas intuições são violadas. O *Rule Engine* é a ponte entre linguagem natural e detecção estruturada.

5.2 Exemplos de Regras

Gestor: "Quero ser alertado quando a prateleira inferior de qualquer zona estiver mais de 30% vazia."

Gestor: "Na zona Z_S1, se não houver produtos de laticínios visíveis, é crítico e preciso de saber imediatamente."

Gestor: "Quando o fill rate de uma prateleira cair abaixo de 60% entre as 10h e as 13h, avisa-me mas não é urgente."

Gestor: "Se um produto estiver tombado, considera sempre severidade alta."

5.3 Schema de Configuração de Regra

O LLM deve converter cada regra em linguagem natural para o seguinte schema JSON:

```
{
  "rule_id": "RULE_001",
  "created_at": "2025-03-17T14:30:00Z",
  "natural_language": "texto original da regra",
  "description": "reformulação clara e inequívoca em português formal",
  "conditions": {
    "zone_filter": ["Z_S1", "Z_S3"],
    "time_filter": {"hours_start": 10, "hours_end": 13},
    "issue_types": ["empty_shelf", "damaged"],
    "severity_threshold": "low|medium|high",
    "fill_rate_threshold": 0.6,
    "location_filter": "bottom|middle|top|any"
  },
  "action": {
    "alert_level": "info|warning|critical",
    "notification_message": "template da mensagem quando a regra dispara"
  },
  "validation": {
    "is_valid": true,
    "ambiguities": ["lista de aspectos não claros"],
    "assumptions": ["lista de pressupostos assumidos na conversão"]
  }
}
```

5.4 Validação e Clarificação

Quando a regra em linguagem natural é ambígua ou incompleta, o sistema não deve assumir silenciosamente, deve identificar as ambiguidades, listá-las no campo

validation.ambiguities, e perguntar ao gestor como deseja resolver cada uma antes de guardar a regra.

Exemplo: o gestor escreve "Avisa-me quando a prateleira estiver vazia." O sistema deve responder identificando as ambiguidades: (1) "vazia" significa 0% ou abaixo de uma percentagem? (2) que nível de urgência? (3) aplica-se a todas as zonas ou a zonas específicas?

5.5 Execução das Regras

Após cada inspeção, o sistema percorre todas as regras guardadas e verifica quais disparam face aos resultados. Para cada regra que dispara, gera uma notificação usando o template definido na configuração, preenchido com os dados concretos da inspeção. O executor de regras deve produzir logs de execução (que regras foram verificadas, quais dispararam e porquê).

6. Componente 3: RAG Memory

6.1 O Problema

Sem memória, cada inspeção é analisada em isolamento. Com memória, o sistema sabe que a zona Z_S1 esteve vazia três vezes nas últimas duas semanas sempre à terça-feira de tarde, ou que um determinado produto foi encontrado na posição errada em 7 das últimas 10 inspeções da zona Z_S3. Este contexto histórico transforma uma deteção de evento num padrão com implicações operacionais.

6.2 O que é Indexado

Cada inspeção concluída gera um inspection record indexado na base de dados vetorial. O campo summary é gerado pela LLM e é o texto que vai ter *embeddings* e ser indexado. O aluno deve conceber prompts que gerem *summaries* ricos em termos semanticamente relevantes para recuperação futura:

Mau summary: "prateleira com problemas." Bom summary: "prateleira inferior da zona Z_S3 com fill rate de 72%, produto de limpeza (detergente líquido) fora de posição na secção central, embalagem danificada detetada no lado direito, terça-feira 15h."

6.3 Stack Tecnológico

- *Embeddings*: sentence-transformers/paraphrase-multilingual-MiniLM-L12-v2 (local, gratuito, suporta português) ou equivalente da HuggingFace Inference API gratuita.
- *Vector store*: ChromaDB (local, persistente em disco, sem servidor) ou FAISS.
- *Retrieval*: similaridade de cosseno com top-k configurável (k=3 por defeito).

```
pip install chromadb sentence-transformers
# ChromaDB persiste em disco por defeito:
client = chromadb.PersistentClient(path="./vectorstore")
```

6.4 Queries ao Sistema de Memória

O sistema deve suportar queries em linguagem natural, traduzidas internamente em pesquisas na vector store. Exemplos obrigatórios a suportar:

- "Quando foi a última vez que a zona Z_S1 teve problemas de prateleira vazia?"
- "Que zonas tiveram mais issues de planograma nas últimas 2 semanas?"
- "Existe algum padrão nos problemas detetados às sextas-feiras à tarde?"
- "Que regras foram mais frequentemente disparadas este mês?"

Para cada query, o sistema recupera os k registos mais relevantes da vector store, constrói um contexto aumentado, e usa a LLM para sintetizar uma resposta com referência explícita às inspeções recuperadas (inspection_id, data).

6.5 Estratégia de Chunking

O aluno deve justificar a sua estratégia de chunking. Há pelo menos três abordagens possíveis, com trade-offs diferentes:

- Record completo como chunk único: simples, mas o vetor representa uma média de todo o conteúdo.
- Por issue: cada issue detetado é indexado separadamente, com metadados da inspeção pai, o que permite recuperação granular mas aumenta o índice.
- Híbrido: o summary da inspeção é indexado como chunk principal, com metadata estruturada (zona, data, fill rate, status) para filtragem pre-retrieval.

A escolha e justificação é parte da avaliação. O aluno deve testar pelo menos duas abordagens e comparar o Recall@3 sobre um conjunto de queries com ground truth definido. Podem ser contempladas técnicas como o TurboQuant da Google para redução do tamanho da KV-cache.

6.6 Integração com Dados de Trajectória (opcional, recompensada)

Se o aluno integrar os dados de trajectória do Projeto 1, a vector store pode ser enriquecida com contexto de afluência. Por exemplo: uma inspeção que deteta prateleira vazia na zona Z_S1 às 15h pode ser contextualizada com o facto de Z_S1 ter tido afluência 40% acima da média nesse período, sugerindo que a causa é falta de stock por procura elevada e não falha de reposição. Esta integração não é obrigatória mas é avaliada como componente de qualidade.

7. Componente 4: Report Generator

Para cada sessão de inspeção, o sistema deve gerar automaticamente um Inspection Report em Markdown com as seguintes secções obrigatórias:

1. Sumário executivo (máx. 150 palavras) - estado geral da loja nesta sessão. Quantas zonas inspecionadas, quantos issues críticos, quantos warnings. Linguagem directa e acionável.
2. Problemas por zona - para cada zona com problemas/incidentes: lista de problemas, severidade, fill rate, e comparação com histórico recuperado do RAG.
3. Regras disparadas - que regras foram activadas, com que dados, e que ação foi gerada.

4. Contexto histórico relevante - padrões passados recuperados do RAG com referências explícitas (inspection_id, data).
5. Recomendações - máximo 5 ações concretas, ordenadas por urgência, cada uma específica o suficiente para ser executável sem interpretação adicional.
6. Integração com trajetória (se implementada) - correlação entre issues detetados e padrões de afluência na zona correspondente no mesmo período.

8. Componente 5: Interface Conversacional

O sistema deve expor uma interface em linha de comandos (CLI) ou Streamlit com os seguintes modos de operação:

```
# Modo de inspeção
> inspect Z_S3 --image shelf_photo.jpg
> inspect all --images-dir ./today_photos/

# Modo de definição de regras
> add rule "Avisa-me quando a prateleira inferior estiver mais de 40% vazia"
> list rules
> delete rule RULE_003
> test rule RULE_001 --image shelf_photo.jpg

# Modo de consulta histórica
> history "quais as zonas com mais problemas esta semana?"
> history "ultima vez que Z_S1 teve fill rate abaixo de 50%?"
> compare Z_S1 Z_S3 --period "last 7 days"

# Modo de relatório
> report --session today
> report --zone Z_S3 --period "last 14 days"
```

A interface deve manter estado de sessão (regras carregadas, histórico de inspeções) e deve responder de forma informativa quando o utilizador faz algo inválido, nunca com stack traces expostos ao utilizador.

9. Avaliação do Sistema

9.1 Harness de Avaliação

O aluno deve construir um harness de avaliação executável com um único comando:

```
python evaluate.py --images-dir test_images/ --output evaluation_report.json
```

O professor fornece test_images/ no momento da defesa — 10 imagens não vistas com ground truth de issues anotado manualmente.

9.2 Métricas Obrigatórias

Análise visual:

Métrica	Descrição
Issue Detection Rate	% de issues do ground truth correctamente identificados (recall)
False Positive Rate	% de issues reportados que não existem no ground truth
Severity Accuracy	% de issues com severidade correctamente classificada
JSON Parse Rate	% de respostas do modelo que são JSON válido parseável
Hallucination Rate	% de afirmações no campo description não verificáveis na imagem

Avaliação do RAG:

Métrica	Descrição
Recall@3	% de queries onde o documento relevante está nos top-3 resultados
Faithfulness	% de afirmações na resposta RAG suportadas pelos chunks recuperados
Answer Relevance	Avaliado por LLM-as-judge: a resposta responde à query?

Avaliação do Rule Engine:

Métrica	Descrição
Rule Parse Rate	% de regras em linguagem natural convertidas em JSON válido
Rule Correctness	% de regras convertidas que executam correctamente sobre dados sintéticos
Ambiguity Detection	% de regras ambíguas correctamente identificadas como tal

9.3 LLM-as-Judge

Para avaliação qualitativa dos relatórios e respostas RAG, o aluno deve implementar um avaliador automático usando o próprio Gemini Flash como juiz. O avaliador recebe o output do sistema e um critério de avaliação e retorna uma pontuação com justificação. O aluno deve documentar os prompts do LLM-as-judge e analisar em que casos o juiz concorda e discorda da sua própria avaliação humana. Esta meta-análise é componente do relatório.

10. Entregáveis

10.1 Código

Repositório com a seguinte estrutura:

```
tp2/
├── README.md
├── requirements.txt
├── .env.example          - variáveis de ambiente (GEMINI_API_KEY, etc.)
├── data/
│   ├── images/          - dataset de imagens (ou script de download)
│   ├── inspections/     - inspection records gerados
│   └── rules/            - regras persistidas
├── src/
└── shelf_inspector.py
```

```
| | rule_engine.py
| | rag_memory.py
| | report_generator.py
| | interface.py
| prompts/
| | *.txt           - todos os prompts versionados
| vectorstore/      - ChromaDB persistente (gerado em runtime)
| cache/            - cache de resultados de API
| evaluate.py
```

10.2 Relatório Técnico

Entre 6 e 12 páginas (excluindo anexos), formato IEEE double column, com as seguintes secções obrigatórias:

7. Introdução e arquitectura: visão geral do sistema, diagrama de arquitectura, decisões de design de alto nível.
8. Dataset: origem, distribuição, limitações e como afectam a avaliação.
9. Análise visual: prompts desenvolvidos, comparação das 3 estratégias com resultados quantitativos, análise de casos de falha.
10. Rule Engine: exemplos de conversão correcta e incorrecta, estratégia de deteção de ambiguidades, limitações.
11. RAG: estratégia de chunking escolhida com comparação de alternativas, métricas de retrieval, análise de falhas de recuperação.
12. Avaliação: resultados do harness, análise do LLM-as-judge, limitações honestas.
13. Integração com Projeto 1 (se implementada): descrição e evidência de melhoria qualitativa.
14. Conclusão: o que funcionou, o que não funcionou, o que faria de diferente.

Anexos (não contam para o limite de páginas): todos os prompts usados; exemplos de inspection records; exemplos de regras convertidas; report semanal gerado.

11. Orientações Técnicas

- Chave de API Gemini: criar conta em <https://aistudio.google.com> → Get API Key. É gratuita, sem cartão de crédito. Nunca commitar a chave no repositório — usar variáveis de ambiente via .env e python-dotenv.
- Gestão de quota: com 1500 req/dia, um dataset de 50 imagens x 3 estratégias de prompting = 150 chamadas de desenvolvimento. Há margem confortável para experimentação, mas o cache de resultados é obrigatório.
- Imagens sem GPU: o processamento de imagens é feito inteiramente pelo Gemini Flash na cloud, o laptop do aluno não faz inferência de visão, apenas chama a API. Não é necessária GPU.
- Reprodutibilidade: Gemini Flash não tem parâmetro de seed exposto. Para garantir reprodutibilidade mínima, usar temperature=0 nos testes e documentar que outputs podem variar ligeiramente entre execuções.

12. Regras e Integridade Académica

- Projecto individual. Todo o código deve ser escrito pelo aluno.

- A chave de API é pessoal e não deve ser partilhada. O uso indevido da chave de outro colega é considerado fraude.
- A utilização de ferramentas de geração de código por IA não é proibida, mas a defesa oral testará a compreensão de cada decisão de design e cada linha de código relevante.
- A defesa oral é obrigatória. Não comparecimento resulta em nota zero.
- O sistema deve executar sem erros críticos. Sistemas que falhem durante a demonstração ao vivo recebem no máximo 50% da cotação.
- O dataset de imagens deve respeitar as licenças das fontes. O relatório deve documentar a origem e licença de cada conjunto de imagens.